SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1715**

NAME: SARANYA.R.A

REG NO: 192572052

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*(BL3)*

Assessment Tool Weightage: 20%

QUESTIONS: 3 **Duration : 45 Minutes**

Total Marks:30

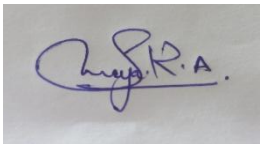
Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I Saranya.R.A (192572052) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A

Constraint-Based Problem Solving

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that:

   Constraints:

   i. D1 cannot work Night shift
   ii. D2 must work before D3 (Morning < Afternoon < Night)
   iii. Only one doctor per shift
   iv. D3 cannot work Morning
   v. All doctors must be assigned exactly one shift

   Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

   Constraints:

   i. Movement allowed: Up, Down, Left, Right
   ii. Cost of each move = 1
   iii. Cannot pass through obstacles

   Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost**.**

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right
- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation
- The robot must update its knowledge dynamically (online search scenario)

Tasks:

a) Analyze all the given constraints and explain how they affect the robot's decision-making process.

b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.

c) Demonstrate how the robot applies AI concepts such as:

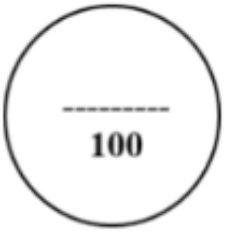
- Intelligent agents
- Rationality
- Environment type

d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Criteria	Weightage	Q1 (10)	Q2 (10)	Q3 (10)	Total Marks (30)
Problem Understanding	25% (2.5)	/2.5	/2.5	/2.5	/7.5
Constraint Analysis	25% (2.5)	/2.5	/2.5	/2.5	/7.5
Solution Development	25% (2.5)	/2.5	/2.5	/2.5	/7.5
Application of Concepts	15% (1.5)	/1.5	/1.5	/1.5	/4.5
Justification	10% (1)	/1	/1	/1	/3
Total per Question	10 Marks	/10	/10	/10	/30



ANSWER

1. Hospital Doctor–Shift Assignment Using Backtracking Search

Problem

There are 3 doctors and 3 shifts:

- Doctors: **D1, D2, D3**
- Shifts: **Morning (M), Afternoon (A), Night (N)**

Constraints

1. **D1 ≠ Night**
2. **D2 must work before D3**
3. **Only one doctor per shift**
4. **D3 ≠ Morning**
5. Every doctor must receive exactly one shift.

Since the order is:

Morning < Afternoon < Night

we need to assign D2 to an earlier shift than D3.

Backtracking Search

We assign shifts one doctor at a time and check constraints after every assignment.

Step 1: Assign D1

Try:

D1 → Morning

Constraint check:

- D1 is not Night → **Satisfied**

- No other doctor is assigned Morning → **Satisfied**

Current assignment:

{D1 = Morning}

Step 2: Assign D2

Try:

D2 → Morning

Constraint check:

- Morning is already occupied by D1 → **Violated**

Therefore:

Backtrack

Reject:

{D1 = Morning, D2 = Morning}

Step 3: Try another shift for D2

Try:

D2 → Afternoon

Constraint check:

- Afternoon is free → **Satisfied**
- D2 must be before D3 → D3 must be after Afternoon
- Therefore, D3 can only take **Night**

Current assignment:

{D1 = Morning, D2 = Afternoon}

Step 4: Assign D3

Try:

D3 → Morning

Constraint check:

- Morning is already occupied → **Violated**
- D3 cannot work Morning → **Violated**

Therefore:

Backtrack

Step 5: Try D3 → Afternoon

Constraint check:

- Afternoon is already occupied by D2 → **Violated**

Therefore:

Backtrack

Step 6: Try D3 → Night

Constraint check:

- Night is free → **Satisfied**
- D3 is not assigned Morning → **Satisfied**
- D2 = Afternoon comes before D3 = Night → **Satisfied**

All constraints are satisfied.

Search Tree

Start

```
│
├── D1 = Morning ✓
│   │
│   ├── D2 = Morning ✗
│   │       └── Shift already occupied → Backtrack
│   │
│   └── D2 = Afternoon ✓
│       │
│       ├── D3 = Morning ✗
│       │       └── Occupied + D3 cannot Morning → Backtrack
│       │
│       ├── D3 = Afternoon ✗
│       │       └── Shift already occupied → Backtrack
│       │
│       └── D3 = Night ✓
│               └── All constraints satisfied
```

Final Valid Schedule

Doctor	Shift
D1	**Morning**
D2	**Afternoon**
D3	**Night**

Final Answer

D1 → Morning, D2 → Afternoon, D3 → Night

Total assignment cost is not specified, so the objective is simply to find a valid assignment. The above schedule satisfies **all five constraints**.

---

2. Robot Grid Navigation Using BFS

Problem

The robot must move from **S (Start)** to **G (Goal)** in a **5×5 grid**.

To demonstrate BFS, consider the following grid:

	C1	C2	C3	C4	C5
R1	S	.	#	.	.
R2	#	.	#	.	#
R3	.	.	.	.	.
R4	.	#	#	#	.
R5	.	.	.	G	.

Where:

- S = Start
- G = Goal
- . = Free cell
- # = Obstacle

Start:

S = (1,1)

Goal:

G = (5,4)

Movement is allowed only:

Up, Down, Left, Right

Each movement has cost:

1

BFS Principle

BFS explores nodes level by level.

The queue follows **FIFO (First In, First Out)** order.

The starting node has distance 0.

Step-by-Step BFS Expansion

Initial State

Queue = [(1,1)]

Visited = {(1,1)}

Expansion 1: (1,1)

Possible neighbors:

- Up → Outside grid
- Down → (2,1) = Obstacle
- Left → Outside grid
- Right → (1,2) = Free

Add:

(1,2)

Queue = [(1,2)]

Distance:

(1,1) = 0

(1,2) = 1

Expansion 2: (1,2)

Possible neighbors:

- Up → Outside
- Down → (2,2) = Free
- Left → (1,1) = Already visited
- Right → (1,3) = Obstacle

Add:

(2,2)

Queue = [(2,2)]

Distance:

(2,2) = 2

Expansion 3: (2,2)

Possible neighbors:

- Up → (1,2) = Visited
- Down → (3,2) = Free
- Left → (2,1) = Obstacle
- Right → (2,3) = Obstacle

Add:

(3,2)

Queue = [(3,2)]

Distance:

(3,2) = 3

Expansion 4: (3,2)

Possible neighbors:

- Up → (2,2) = Visited
- Down → (4,2) = Obstacle
- Left → (3,1) = Free
- Right → (3,3) = Free

Add:

(3,1), (3,3)

Queue = [(3,1), (3,3)]

Expansion 5: (3,1)

Possible neighbors:

- Up → (2,1) = Obstacle

- Down → (4,1) = Free
- Left → Outside
- Right → (3,2) = Visited

Add:

(4,1)

Queue = [(3,3), (4,1)]

Expansion 6: (3,3)

Possible neighbors:

- Up → (2,3) = Obstacle
- Down → (4,3) = Obstacle
- Left → (3,2) = Visited
- Right → (3,4) = Free

Add:

(3,4)

Queue = [(4,1), (3,4)]

Expansion 7: (4,1)

Possible neighbors:

- Up → (3,1) = Visited
- Down → (5,1) = Free
- Left → Outside
- Right → (4,2) = Obstacle

Add:

(5,1)

Queue = [(3,4), (5,1)]

Expansion 8: (3,4)

Possible neighbors:

- Up → (2,4) = Free
- Down → (4,4) = Obstacle
- Left → (3,3) = Visited
- Right → (3,5) = Free

Add:

(2,4), (3,5)

Queue = [(5,1), (2,4), (3,5)]

Expansion 9: (5,1)

Possible neighbors:

- Up → (4,1) = Visited

- Down → Outside
- Left → Outside
- Right → (5,2) = Free

Add:

(5,2)

Queue = [(2,4), (3,5), (5,2)]

Expansion 10: (2,4)

Possible neighbors:

- Up → (1,4) = Free
- Down → (3,4) = Visited
- Left → (2,3) = Obstacle
- Right → (2,5) = Obstacle

Add:

(1,4)

Queue = [(3,5), (5,2), (1,4)]

Expansion 11: (3,5)

Possible neighbors:

- Up → (2,5) = Obstacle
- Down → (4,5) = Free
- Left → (3,4) = Visited
- Right → Outside

Add:

(4,5)

Queue = [(5,2), (1,4), (4,5)]

Expansion 12: (5,2)

Possible neighbors:

- Up → (4,2) = Obstacle
- Down → Outside
- Left → (5,1) = Visited
- Right → (5,3) = Free

Add:

(5,3)

Queue = [(1,4), (4,5), (5,3)]

Expansion 13: (1,4)

Possible neighbors:

- Up → Outside

- Down → (2,4) = Visited
- Left → (1,3) = Obstacle
- Right → (1,5) = Free

Add:

(1,5)

Queue = [(4,5), (5,3), (1,5)]

Expansion 14: (4,5)

Possible neighbors:

- Up → (3,5) = Visited
- Down → (5,5) = Free
- Left → (4,4) = Obstacle
- Right → Outside

Add:

(5,5)

Queue = [(5,3), (1,5), (5,5)]

Expansion 15: (5,3)

Possible neighbors:

- Up → (4,3) = Obstacle
- Down → Outside
- Left → (5,2) = Visited
- Right → (5,4) = **Goal**

Goal found.

Therefore, BFS stops.

BFS Expansion Summary

Step	Expanded Node	Newly Added Nodes	Queue After Expansion
1	(1,1)	(1,2)	(1,2)
2	(1,2)	(2,2)	(2,2)
3	(2,2)	(3,2)	(3,2)
4	(3,2)	(3,1), (3,3)	(3,1),(3,3)
5	(3,1)	(4,1)	(3,3),(4,1)
6	(3,3)	(3,4)	(4,1),(3,4)
7	(4,1)	(5,1)	(3,4),(5,1)
8	(3,4)	(2,4),(3,5)	(5,1),(2,4),(3,5)
9	(5,1)	(5,2)	(2,4),(3,5),(5,2)
10	(2,4)	(1,4)	(3,5),(5,2),(1,4)

Step	Expanded Node	Newly Added Nodes	Queue After Expansion
11	(3,5)	(4,5)	(5,2),(1,4),(4,5)
12	(5,2)	(5,3)	(1,4),(4,5),(5,3)
13	(1,4)	(1,5)	(4,5),(5,3),(1,5)
14	(4,5)	(5,5)	(5,3),(1,5),(5,5)
15	(5,3)	**(5,4) Goal**	Goal Found

Optimal Path

Tracing the parent nodes backward:

(1,1)
↓
(1,2)
↓
(2,2)
↓
(3,2)
→
(3,3)
→
(3,4)
↓
(3,5)
↓
(4,5)
↓
(5,5)
←
(5,4) = G

Therefore:

S → (1,2) → (2,2) → (3,2) → (3,3) → (3,4) → (3,5) → (4,5) → (5,5) → G

Optimal Cost

There are **9 moves**.

Total cost = 9 × 1 = 9

Final Result

Optimal Path Cost = 9

BFS guarantees the shortest path here because **every movement has the same cost of 1**.

Note: BFS uses a normal FIFO queue rather than a priority queue. If the term "priority queue" is required in the question, that structure belongs to **Uniform Cost Search (UCS)**. For this problem, because every edge cost is 1, BFS and UCS produce the same optimal path cost.

---

3. Autonomous Rescue Robot in a Partially Observable Building

Scenario

An autonomous rescue robot must find a survivor at **G**, starting from **S**. The robot does not initially know the complete state of the building.

It can observe only nearby cells, and some areas may be blocked or risky.

The robot must therefore make decisions while continuously updating its knowledge.

3(a). Analysis of Constraints and Their Effects

1. Movement Constraint

The robot can move only:

- Up
- Down
- Left
- Right

It cannot move diagonally.

Effect

The robot must consider only four possible actions at each location. This reduces the available action space and prevents diagonal shortcuts.

---

2. Obstacles

Blocked cells cannot be entered.

Effect

The robot must detect blocked paths and search for alternative routes. Since the robot initially has incomplete information, a path that appears possible may later turn out to be blocked.

---

3. Limited Sensing

The robot can observe only adjacent cells.

Effect

The robot cannot build a complete map immediately. It must:

1. Observe the nearby environment.
2. Update its internal map.
3. Select the next action.
4. Move to the next location.
5. Sense again.
6. Repeat the process.

This makes the problem **partially observable** and requires an **online search approach**.

---

4. Normal Movement Cost

Every ordinary movement costs:

1

The robot should avoid unnecessary movement because extra movements increase the total cost.

---

5. Risky Zone Cost

Entering a risky zone has an additional cost of:

2

Therefore:

Risky cell movement cost = 1 + 2 = 3

Effect

The shortest path in terms of distance may not be the cheapest path.

For example:

Path A: 4 normal moves

Cost = 4 × 1 = 4

Path B: 3 normal moves + 1 risky move

Cost = 3 + 3 = 6

Although Path B is shorter in terms of number of moves, **Path A is preferred** because its total cost is lower.

---

6. Dynamic Knowledge

The robot's map changes as it discovers new information.

Effect

The robot cannot blindly follow a fixed route. If a newly discovered obstacle blocks the planned path, the robot must update its map and re-plan.

---

7. Safety Requirement

The robot must reach the survivor while avoiding unnecessary risks.

Effect

The decision is based on both:

- **Path cost**
- **Safety**

Therefore, the robot should select the **lowest-cost safe route based on currently known information**.

3(b). Proposed Solution: Online Search with Uniform Cost Search

A suitable solution is an **Online Search Agent combined with Uniform Cost Search (UCS)**.

UCS is appropriate because the movement costs are not uniform:

- Normal movement = **1**
- Risky movement = **3**

Unlike BFS, UCS considers the **actual cumulative path cost**.

Working Procedure

Step 1: Initialize

The robot starts at:

S

It initially knows only the information available through its sensors.

Known Map = Local information around S

Current Position = S

Goal = G

Step 2: Sense the Environment

The robot observes adjacent cells.

It identifies:

- Free cells
- Obstacles
- Risky cells
- Possible movement directions

Step 3: Update Internal Map

The robot stores the newly discovered information.

For example:

S → Free

Right → Free

Down → Obstacle

The robot now knows that moving downward is impossible and must consider another direction.

Step 4: Calculate Path Costs

For each possible movement:

Normal cell:

New Cost = Current Cost + 1

Risky cell:

New Cost = Current Cost + 3

The robot maintains the cumulative cost of reaching each discovered cell.

Step 5: Select Lowest-Cost Candidate

UCS selects the unexplored node with the smallest accumulated cost.

For example:

Node A → Cost 5

Node B → Cost 7

Node C → Cost 4

UCS selects:

Node C

because it has the lowest cost.

Step 6: Move and Sense Again

After selecting an action, the robot moves to the chosen cell.

It then performs sensing again.

The newly observed information is added to the map.

Step 7: Re-plan When Necessary

Suppose the robot planned:

S → A → B → C → G

But after reaching A, it discovers that:

B = Obstacle

The robot does not continue with the old plan.

Instead, it:

1. Marks B as blocked.
2. Updates its map.
3. Removes the invalid route.
4. Runs the search again.
5. Selects the lowest-cost known safe alternative.

Step 8: Reach the Survivor

The process continues until:

Current Position = G

The robot then stops searching and performs the rescue operation.

Online Search Flow

Start at S

|

▼

Sense nearby cells

|

▼

Update internal map

|

▼

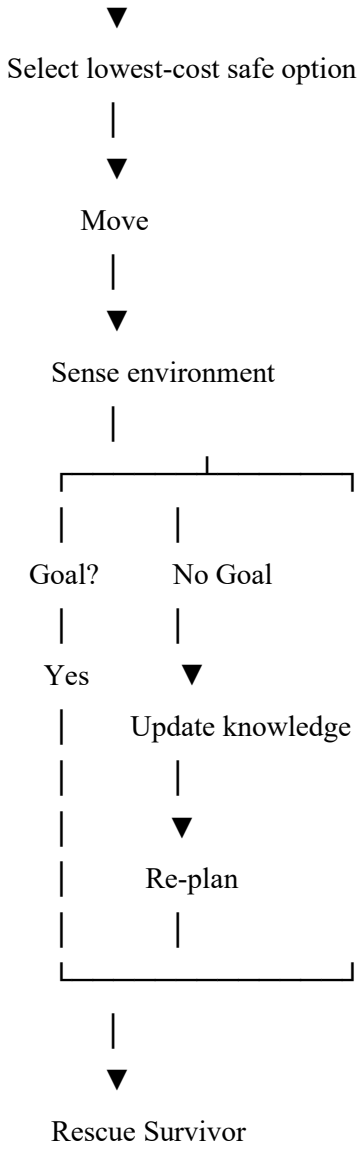
Generate possible actions

|

▼

Calculate cumulative cost

|



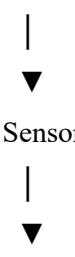
3(c). Application of AI Concepts

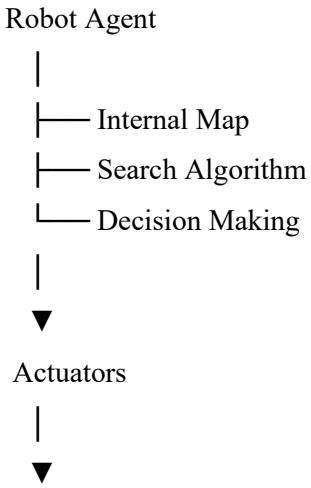
1. Intelligent Agent

The rescue robot is an **intelligent agent** because it:

- Perceives the environment through sensors.
- Maintains information about the environment.
- Chooses actions.
- Executes movements.
- Updates its knowledge.
- Re-plans when conditions change.

The agent can be represented as:





The robot follows the cycle:

Perceive → Think → Act → Perceive Again

---

2. Rationality

A rational agent chooses an action that gives the best expected result based on its available information.

For this rescue robot, rational behavior means:

- Avoid blocked cells.
- Prefer safe routes.
- Minimize total movement cost.
- Consider the extra cost of risky areas.
- Use newly discovered information.
- Re-plan when the environment differs from expectations.
- Continue toward the survivor.

The robot does **not** simply choose the route with the fewest steps. It chooses the route with the **lowest appropriate cost while maintaining safe navigation**.

---

3. Environment Type

The environment can be classified as follows:

Property	**Classification**	**Reason**
Observability	**Partially Observable**	Robot can sense only adjacent cells
Agents	**Single-Agent**	One rescue robot performs the search
Determinism	**Partially Deterministic / Uncertain**	The robot may discover unexpected obstacles
Episodic/Sequential	**Sequential**	Current actions affect future decisions
Static/Dynamic	**Dynamic or Changing**	Information about the environment is revealed over time
Discrete/Continuous	**Discrete**	Grid cells and movements are discrete
Known/Unknown	**Initially Unknown**	Complete map is not available initially

Therefore, the overall environment is best described as a:

Partially observable, sequential, dynamic, discrete, initially unknown environment.

3(d). Justification for the Chosen Approach

The combination of **Online Search Agent + Uniform Cost Search** is suitable for this problem for several reasons.

Reason 1: Partial Observability

The robot cannot see the entire building.

An online agent can make decisions using the information currently available and update its knowledge as it moves.

Reason 2: Different Movement Costs

Normal cells cost **1**, while risky cells cost **3**.

BFS is not ideal because BFS assumes every action has equal cost.

UCS is more suitable because it selects paths based on **cumulative cost**.

Reason 3: Dynamic Re-planning

If the robot discovers a previously unknown obstacle, it can update its internal map and search for another route.

This is essential in a disaster environment.

Reason 4: Cost Minimization

UCS ensures that the robot considers the actual cost of reaching a location rather than simply counting the number of movements.

Thus, a slightly longer safe route may be selected over a shorter route containing expensive risky zones.

Reason 5: Safe Navigation

The robot can treat known obstacles as unavailable and risky cells as higher-cost states.

This allows the search strategy to balance:

Safety + Distance + Movement Cost

Final Conclusion

The hospital scheduling problem can be solved using **Backtracking Search**, where invalid assignments are detected early and the algorithm backtracks until it finds:

D1 → Morning, D2 → Afternoon, D3 → Night

For the fully known 5×5 grid with equal movement costs, **BFS** finds the shortest path. For the demonstrated grid, the optimal route has a cost of **9**.

For the autonomous rescue robot, the environment is **partially observable and initially unknown**, so a combination of an **Online Search Agent and Uniform Cost Search** is more appropriate. The robot continuously senses nearby cells, updates its internal map, considers obstacle and risk information, and selects a low-cost safe route. This approach demonstrates the AI concepts of **intelligent agents, rational decision-making, perception, action, dynamic knowledge updating, and search-based planning**.